

# RUNBOOK NGÀY 3 — SPRINT 1:

## AUTHENTICATION & PHÂN QUYỀN

Tài liệu nghiệm thu kỹ thuật thực tế (Post-Mortem & Implementation Record)

### 1. METADATA & CHỈ SỐ KỸ THUẬT

| Hạng mục                            | Thông số chuẩn hóa                                                            |
|-------------------------------------|-------------------------------------------------------------------------------|
| Runtime SDK                         | .NET 10.0 (LTS)                                                               |
| Database Engine                     | PostgreSQL 16+ (Npgsql Provider)                                              |
| Frontend Framework                  | Blazor Server (InteractiveServer Render Mode)                                 |
| UI Component Library                | MudBlazor 8.x                                                                 |
| Authentication Strategy             | Cookie Authentication (ASP.NET Core Identity) qua MVC Controller + JS Interop |
| Thời gian thực tế / Dự kiến         | 230 phút / 120 phút (+110 phút do xử lý Shadow Properties & Column Mapping)   |
| Trạng thái DoD (Definition of Done) | ✅ ĐẠT (7/7 tiêu chí kiểm thử PASS)                                            |

### 2. QUY CHUẨN THỰC THI (PHASE-BY-PHASE)

#### PHASE 1: Cài đặt Dependencies chuẩn hóa theo Project

**Nguyên tắc phân tầng:** Project con không tự động thừa kế toàn bộ transitive references từ project cha khi dùng interface Identity. Phải cài đúng package vào từng project:

```
# 1. Cài Identity EF Core Store vào Data Layer
cd src\Crm.Data
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore --version 10.0.0

# 2. Cài Identity Stores abstractions vào Domain Layer (cho ApplicationUser)
cd ../Crm.Domain
dotnet add package Microsoft.Extensions.Identity.Stores --version 10.0.0
```

```
# 3. Kiểm tra UI Layer (Đã có sẵn MudBlazor 8.x)
cd ..\Crm.Web
dotnet restore
```

## PHASE 2: Triển khai Domain Entities

### 2.1 File src\Crm.Domain\Entities\ApplicationUser.cs

Tách biệt thông tin đăng nhập khỏi nghiệp vụ:

```
using Microsoft.AspNetCore.Identity;

namespace Crm.Domain.Entities;

public class ApplicationUser : IdentityUser<Guid>
{
    public ApplicationUser()
    {
        Id = Guid.NewGuid();
    }
}
```

### 2.2 File src\Crm.Domain\Entities\UserProfile.cs

Bắt buộc khai báo navigation property chuẩn để triệt tiêu lỗi tạo cột ẩn (Shadow Property):

```
namespace Crm.Domain.Entities;

public class UserProfile
{
    public Guid UserId { get; set; }
    public string FullName { get; set; } = string.Empty;
    public string? EmployeeCode { get; set; }
    public string? Phone { get; set; }
    public string? AvatarUrl { get; set; }
    public string RoleCode { get; set; } = "SALES";
    public int? TeamId { get; set; }
    public decimal MonthlyTarget { get; set; } = 0;
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;

    // Navigation properties (Bắt buộc gán null! để tránh warning nullable & shadow property)
```

```

    public virtual ApplicationUser User { get; set; } = null!;
    public virtual Team? Team { get; set; }
}

```

### 2.3 File src\Crm.Domain\Entities\Team.cs

```

namespace Crm.Domain.Entities;

public class Team
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public Guid? ManagerId { get; set; }
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public virtual ApplicationUser? Manager { get; set; }
    public virtual ICollection<UserProfile> Members { get; set; } = new List<UserProfile>();
}

```

## PHASE 3: Cấu hình AppDbContext (Manual Column Mapping)

**Cảnh báo cốt lõi:** PostgreSQL dùng convention snake\_case, trong khi EF Core mặc định query PascalCase. Bắt buộc phải cấu hình HasColumnName thủ công cho toàn bộ property để tránh lỗi column "UserId" does not exist.

### File src\Crm.Data\AppDbContext.cs:

```

using Crm.Domain.Entities;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace Crm.Data;

public class AppDbContext : IdentityDbContext<ApplicationUser, IdentityRole<Guid>, Guid>
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<UserProfile> UserProfiles => Set<UserProfile>();
    public DbSet<Team> Teams => Set<Team>();
}

```

```

protected override void OnModelCreating(ModelBuilder builder)
{
    base.OnModelCreating(builder);

    // ===== MAP USER_PROFILES =====
    builder.Entity<UserProfile>(entity =>
    {
        entity.ToTable("user_profiles");
        entity.HasKey(u => u.UserId);

        entity.Property(u => u.UserId).HasColumnName("user_id");
        entity.Property(u =>
u.FullName).HasColumnName("full_name").IsRequired().HasMaxLength(100);
        entity.Property(u =>
u.EmployeeCode).HasColumnName("employee_code").HasMaxLength(20);
        entity.Property(u => u.Phone).HasColumnName("phone").HasMaxLength(20);
        entity.Property(u => u.AvatarUrl).HasColumnName("avatar_url").HasMaxLength(500);
        entity.Property(u =>
u.RoleCode).HasColumnName("role_code").IsRequired().HasMaxLength(20);
        entity.Property(u => u.TeamId).HasColumnName("team_id");
        entity.Property(u =>
u.MonthlyTarget).HasColumnName("monthly_target").HasColumnType("numeric(18,2)");
        entity.Property(u => u.IsActive).HasColumnName("is_active");
        entity.Property(u => u.CreatedAt).HasColumnName("created_at");
        entity.Property(u => u.UpdatedAt).HasColumnName("updated_at");

        // Quan hệ 1-1 với ApplicationUser: Phải trở qua Navigation Property
        entity.HasOne(u => u.User)
            .WithOne()
            .HasForeignKey<UserProfile>(u => u.UserId)
            .OnDelete(DeleteBehavior.Cascade);

        entity.HasOne(u => u.Team)
            .WithMany(t => t.Members)
            .HasForeignKey(u => u.TeamId)
            .OnDelete(DeleteBehavior.SetNull);
    });

    // ===== MAP TEAMS =====
    builder.Entity<Team>(entity =>
    {
        entity.ToTable("teams");
        entity.HasKey(t => t.Id);
    });

```

```

entity.Property(t => t.Id).HasColumnName("id");
entity.Property(t => t.Name).HasColumnName("name").IsRequired().HasMaxLength(100);
entity.Property(t => t.ManagerId).HasColumnName("manager_id");
entity.Property(t => t.IsActive).HasColumnName("is_active");
entity.Property(t => t.CreatedAt).HasColumnName("created_at");

entity.HasOne(t => t.Manager)
    .WithMany()
    .HasForeignKey(t => t.ManagerId)
    .OnDelete(DeleteBehavior.SetNull);
});
}
}

```

## PHASE 4: Cơ sở dữ liệu & Ràng buộc Khóa ngoại (DDL Scripts)

Thực hiện toàn bộ script thông qua file (-f), không truyền chuỗi inline qua PowerShell (-c) vì ký tự " sẽ bị strip gây lỗi SQL syntax.

### 4.1 Tạo 7 bảng Identity: docs/db/02\_identity\_tables.sql

Cột định danh phải giữ nguyên bọc kép "Id" để khớp với quy ước mặc định của ASP.NET Core Identity.

```

CREATE TABLE IF NOT EXISTS "AspNetRoles" (
    "Id"          UUID PRIMARY KEY,
    "Name"        VARCHAR(256),
    "NormalizedName" VARCHAR(256),
    "ConcurrencyStamp" TEXT
);

```

```

CREATE UNIQUE INDEX IF NOT EXISTS "RoleNameIndex"
ON "AspNetRoles" ("NormalizedName")
WHERE "NormalizedName" IS NOT NULL;

```

```

CREATE TABLE IF NOT EXISTS "AspNetUsers" (
    "Id"          UUID PRIMARY KEY,
    "UserName"     VARCHAR(256),
    "NormalizedUserName" VARCHAR(256),
    "Email"        VARCHAR(256),
    "NormalizedEmail" VARCHAR(256),
    "EmailConfirmed" BOOLEAN NOT NULL DEFAULT FALSE,

```

```
"PasswordHash"    TEXT,
"SecurityStamp"    TEXT,
"ConcurrencyStamp" TEXT,
"PhoneNumber"      TEXT,
"PhoneNumberConfirmed" BOOLEAN NOT NULL DEFAULT FALSE,
"TwoFactorEnabled" BOOLEAN NOT NULL DEFAULT FALSE,
"LockoutEnd"       TIMESTAMPTZ,
"LockoutEnabled"    BOOLEAN NOT NULL DEFAULT FALSE,
"AccessFailedCount" INT NOT NULL DEFAULT 0
);
```

```
CREATE UNIQUE INDEX IF NOT EXISTS "UserNameIndex" ON "AspNetUsers"
("NormalizedUserName") WHERE "NormalizedUserName" IS NOT NULL;
CREATE INDEX IF NOT EXISTS "EmailIndex" ON "AspNetUsers" ("NormalizedEmail");
```

```
CREATE TABLE IF NOT EXISTS "AspNetUserRoles" (
  "UserId" UUID NOT NULL REFERENCES "AspNetUsers"("Id") ON DELETE CASCADE,
  "RoleId" UUID NOT NULL REFERENCES "AspNetRoles"("Id") ON DELETE CASCADE,
  PRIMARY KEY ("UserId", "RoleId")
);
```

```
CREATE TABLE IF NOT EXISTS "AspNetUserClaims" (
  "Id"      SERIAL PRIMARY KEY,
  "UserId"  UUID NOT NULL REFERENCES "AspNetUsers"("Id") ON DELETE CASCADE,
  "ClaimType" TEXT,
  "ClaimValue" TEXT
);
CREATE INDEX IF NOT EXISTS "IX_AspNetUserClaims_UserId" ON "AspNetUserClaims"
("UserId");
```

```
CREATE TABLE IF NOT EXISTS "AspNetRoleClaims" (
  "Id"      SERIAL PRIMARY KEY,
  "RoleId"  UUID NOT NULL REFERENCES "AspNetRoles"("Id") ON DELETE CASCADE,
  "ClaimType" TEXT,
  "ClaimValue" TEXT
);
CREATE INDEX IF NOT EXISTS "IX_AspNetRoleClaims_RoleId" ON "AspNetRoleClaims" ("RoleId");
```

```
CREATE TABLE IF NOT EXISTS "AspNetUserLogins" (
  "LoginProvider" VARCHAR(128) NOT NULL,
  "ProviderKey"   VARCHAR(128) NOT NULL,
  "ProviderDisplayName" TEXT,
  "UserId"        UUID NOT NULL REFERENCES "AspNetUsers"("Id") ON DELETE CASCADE,
```

```

PRIMARY KEY ("LoginProvider", "ProviderKey")
);
CREATE INDEX IF NOT EXISTS "IX_AspNetUserLogins_UserId" ON "AspNetUserLogins"
("UserId");

CREATE TABLE IF NOT EXISTS "AspNetUserTokens" (
  "UserId"      UUID NOT NULL REFERENCES "AspNetUsers"("Id") ON DELETE CASCADE,
  "LoginProvider" VARCHAR(128) NOT NULL,
  "Name"        VARCHAR(128) NOT NULL,
  "Value"       TEXT,
  PRIMARY KEY ("UserId", "LoginProvider", "Name")
);

```

## 4.2 Tạo hồ sơ nhân sự và 5 FKs: docs/db/03\_user\_profiles.sql

Tất cả các khóa ngoại trở sang User bắt buộc phải dùng cú pháp: REFERENCES "AspNetUsers"("Id").

```

CREATE TABLE IF NOT EXISTS teams (
  id      SERIAL PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  manager_id UUID REFERENCES "AspNetUsers"("Id") ON DELETE SET NULL,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS user_profiles (
  user_id      UUID PRIMARY KEY REFERENCES "AspNetUsers"("Id") ON DELETE CASCADE,
  full_name    VARCHAR(100) NOT NULL,
  employee_code VARCHAR(20) UNIQUE,
  phone        VARCHAR(20),
  avatar_url   VARCHAR(500),
  role_code    VARCHAR(20) NOT NULL DEFAULT 'SALES',
  team_id      INT REFERENCES teams(id) ON DELETE SET NULL,
  monthly_target NUMERIC(18,2) DEFAULT 0,
  is_active    BOOLEAN DEFAULT TRUE,
  created_at   TIMESTAMPTZ DEFAULT NOW(),
  updated_at   TIMESTAMPTZ DEFAULT NOW(),
  CONSTRAINT chk_role_code CHECK (role_code IN ('ADMIN', 'MANAGER', 'SALES',
'ACCOUNTANT'))
);

```

```

-- Liên kết FK từ 5 bảng nghiệp vụ Ngày 2 sangAspNetUsers
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE
constraint_name = 'fk_customers_user') THEN
        ALTER TABLE customers ADD CONSTRAINT fk_customers_user FOREIGN KEY
(assigned_to_user_id) REFERENCES "AspNetUsers"("Id") ON DELETE SET NULL;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE
constraint_name = 'fk_opp_user') THEN
        ALTER TABLE opportunities ADD CONSTRAINT fk_opp_user FOREIGN KEY
(assigned_to_user_id) REFERENCES "AspNetUsers"("Id") ON DELETE SET NULL;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE
constraint_name = 'fk_tasks_user') THEN
        ALTER TABLE sales_tasks ADD CONSTRAINT fk_tasks_user FOREIGN KEY
(assigned_to_user_id) REFERENCES "AspNetUsers"("Id") ON DELETE RESTRICT;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE
constraint_name = 'fk_interactions_user') THEN
        ALTER TABLE interactions ADD CONSTRAINT fk_interactions_user FOREIGN KEY (user_id)
REFERENCES "AspNetUsers"("Id") ON DELETE RESTRICT;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM information_schema.table_constraints WHERE
constraint_name = 'fk_quotes_user') THEN
        ALTER TABLE quotes ADD CONSTRAINT fk_quotes_user FOREIGN KEY
(created_by_user_id) REFERENCES "AspNetUsers"("Id") ON DELETE RESTRICT;
    END IF;
END $$;

```

### 4.3 Script Reset Data khi cần Seeding lại: docs/db/reset\_seed.sql

```

BEGIN;
DELETE FROM user_profiles;
DELETE FROM "AspNetUserRoles";
DELETE FROM "AspNetUserClaims";
DELETE FROM "AspNetUserLogins";
DELETE FROM "AspNetUserTokens";

```

```
DELETE FROM "AspNetUsers";
DELETE FROM "AspNetRoleClaims";
DELETE FROM "AspNetRoles";
DELETE FROM teams;
COMMIT;
```

Thực thi tuần tự:

```
psql -U crm_user -d crm_db -f "docs/db/02_identity_tables.sql"
psql -U crm_user -d crm_db -f "docs/db/03_user_profiles.sql"
```

## PHASE 5: Tự động Seeding Dữ liệu (DbSeeder.cs)

Không sử dụng ILogger<DbSeeder> vì class static không thể làm generic type argument trong C#. Sử dụng ILoggerFactory.CreateLogger("DbSeeder").

**File src\Crm.Data\DbSeeder.cs:**

```
using Crm.Domain.Entities;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace Crm.Data;

public static class DbSeeder
{
    private static readonly string[] Roles = ["ADMIN", "MANAGER", "SALES", "ACCOUNTANT"];

    public static async Task SeedAsync(IServiceProvider services, bool seedDemoData = false)
    {
        using var scope = services.CreateScope();
        var userManager =
            scope.ServiceProvider.GetRequiredService<UserManager<ApplicationUser>>();
        var roleManager =
            scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole<Guid>>>();
        var dbContext = scope.ServiceProvider.GetRequiredService<AppDbContext>();

        var loggerFactory = scope.ServiceProvider.GetRequiredService<ILoggerFactory>();
        var logger = loggerFactory.CreateLogger("DbSeeder");
```

```

// 1. Seed Roles
foreach (var roleName in Roles)
{
    if (!await roleManager.RoleExistsAsync(roleName))
    {
        await roleManager.CreateAsync(new IdentityRole<Guid>
        {
            Id = Guid.NewGuid(),
            Name = roleName,
            NormalizedName = roleName.ToUpperInvariant()
        });
        logger.LogInformation("Role created: {Role}", roleName);
    }
}

// 2. Seed Default Admin
const string adminEmail = "admin@crm.local";
const string adminPassword = "Admin@2026";

var adminUser = await userManager.FindByEmailAsync(adminEmail);
if (adminUser == null)
{
    adminUser = new ApplicationUser
    {
        Id = Guid.NewGuid(),
        UserName = adminEmail,
        Email = adminEmail,
        EmailConfirmed = true,
        LockoutEnabled = true
    };

    var createRes = await userManager.CreateAsync(adminUser, adminPassword);
    if (createRes.Succeeded)
    {
        await userManager.AddToRoleAsync(adminUser, "ADMIN");
        dbContext.UserProfiles.Add(new UserProfile
        {
            UserId = adminUser.Id,
            FullName = "Hệ Thống Quản Trị",
            EmployeeCode = "ADM001",
            RoleCode = "ADMIN",
            IsActive = true
        });
    }
}

```

```

        await dbContext.SaveChangesAsync();
        logger.LogInformation("Admin account created: {Email}", adminEmail);
    }
}

// 3. Seed Demo Users (Chỉ chạy ở Development)
if (seedDemoData)
{
    await SeedDemoUsersAsync(userManager, dbContext, logger);
}
}

private static async Task SeedDemoUsersAsync(
    UserManager<ApplicationUser> userManager,
    AppDbContext dbContext,
    ILogger logger)
{
    var salesTeam = await dbContext.Teams.FirstOrDefaultAsync(t => t.Name == "Đội Kinh
Doanh 01");
    if (salesTeam == null)
    {
        salesTeam = new Team { Name = "Đội Kinh Doanh 01", IsActive = true };
        dbContext.Teams.Add(salesTeam);
        await dbContext.SaveChangesAsync();
    }

    var demoUsers = new[]
    {
        new { Email = "sales1@crm.local", Password = "Sales@2026", Name = "Nguyễn Văn An",
Code = "SAL001", Role = "SALES", Target = 150000000m },
        new { Email = "sales2@crm.local", Password = "Sales@2026", Name = "Trần Thị Bình", Code
= "SAL002", Role = "SALES", Target = 120000000m },
        new { Email = "manager@crm.local", Password = "Manager@2026", Name = "Lê Hoàng
Nam", Code = "MGR001", Role = "MANAGER", Target = 500000000m },
        new { Email = "accountant@crm.local", Password = "Acc@2026", Name = "Phạm Thu Cúc",
Code = "ACC001", Role = "ACCOUNTANT", Target = 0m }
    };

    foreach (var item in demoUsers)
    {
        if (await userManager.FindByEmailAsync(item.Email) != null) continue;

        var user = new ApplicationUser

```

```

{
    Id = Guid.NewGuid(),
    UserName = item.Email,
    Email = item.Email,
    EmailConfirmed = true,
    LockoutEnabled = true
};

var res = await userManager.CreateAsync(user, item.Password);
if (res.Succeeded)
{
    await userManager.AddToRoleAsync(user, item.Role);
    dbContext.UserProfiles.Add(new UserProfile
    {
        UserId = user.Id,
        FullName = item.Name,
        EmployeeCode = item.Code,
        RoleCode = item.Role,
        MonthlyTarget = item.Target,
        TeamId = item.Role == "SALES" ? salesTeam.Id : null,
        IsActive = true
    });

    if (item.Role == "MANAGER" && salesTeam.ManagerId == null)
    {
        salesTeam.ManagerId = user.Id;
    }
    logger.LogInformation("Demo user created: {Email}", item.Email);
}
}
await dbContext.SaveChangesAsync();
}
}

```

## PHASE 6: Đường ống Xác thực & Cấu hình Ứng dụng (Program.cs)

**Quy tắc:** Tạm thời loại bỏ AddApexCharts() ở Ngày 3 để tránh xung đột namespace với MudBlazor.

**File src\Crm.Web\Program.cs:**

```

using Crm.Data;
using Crm.Domain.Entities;

```

```

using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using MudBlazor.Services;

AppContext.SetSwitch("Npgsql.EnableLegacyTimestampBehavior", false);

var builder = WebApplication.CreateBuilder(args);

// 1. Data Connection
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException("Connection string 'DefaultConnection' not found.");

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(connectionString, npgsql => npgsql.EnableRetryOnFailure(3)));

// 2. Identity & Cookies
builder.Services.AddIdentity<ApplicationUser, IdentityRole<Guid>>(options =>
{
    options.Password.RequireDigit = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireUppercase = true;
    options.Password.RequireNonAlphanumeric = true;
    options.Password.RequiredLength = 8;
    options.Password.RequiredUniqueChars = 4;
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
    options.Lockout.MaxFailedAccessAttempts = 5;
    options.Lockout.AllowedForNewUsers = true;
    options.User.RequireUniqueEmail = true;
})
.AddEntityFrameworkStores<AppDbContext>()
.AddDefaultTokenProviders();

builder.Services.ConfigureApplicationCookie(options =>
{
    options.Cookie.Name = "CrmSessionCookie";
    options.Cookie.HttpOnly = true;
    options.Cookie.SameSite = SameSiteMode.Lax;
    options.Cookie.SecurePolicy = CookieSecurePolicy.SameAsRequest;
    options.ExpireTimeSpan = TimeSpan.FromHours(8);
    options.SlidingExpiration = true;
    options.LoginPath = "/login";
    options.LogoutPath = "/logout";
    options.AccessDeniedPath = "/access-denied";

```

```
});
```

```
// 3. UI Services
```

```
builder.Services.AddRazorComponents()  
    .AddInteractiveServerComponents();
```

```
builder.Services.AddMudServices();  
builder.Services.AddCascadingAuthenticationState();  
builder.Services.AddControllers();
```

```
var app = builder.Build();
```

```
// 4. Request Pipeline
```

```
if (!app.Environment.IsDevelopment())  
{  
    app.UseExceptionHandler("/Error", createScopeForErrors: true);  
    app.UseHsts();  
}
```

```
app.UseHttpsRedirection();  
app.UseStaticFiles();
```

```
app.UseAuthentication();  
app.UseAuthorization();  
app.UseAntiforgery();
```

```
app.MapControllers();  
app.MapRazorComponents<Crm.Web.Components.App>()  
    .AddInteractiveServerRenderMode();
```

```
// 5. Database Auto-Seeding
```

```
using (var scope = app.Services.CreateScope())  
{  
    try  
    {  
        var env = app.Services.GetRequiredService<IWebHostEnvironment>();  
        await DbSeeder.SeedAsync(app.Services, seedDemoData: env.IsDevelopment());  
    }  
    catch (Exception ex)  
    {  
        var logger = app.Services.GetRequiredService<ILogger<Program>>();  
        logger.LogError(ex, "Lỗi xảy ra trong quá trình seeding database.");  
    }  
}
```

```
}
```

```
app.Run();
```

## PHASE 7: Blazor Server Cookie Auth Pipeline & Giao diện

### 7.1 File src\Crm.Web\Controllers\AccountController.cs

Xử lý set/clear cookie thực tế qua HTTP POST endpoint:

```
using Crm.Domain.Entities;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;

namespace Crm.Web.Controllers;

[Route("api/account")]
[ApiController]
public class AccountController : ControllerBase
{
    private readonly SignInManager<ApplicationUser> _signInManager;
    private readonly ILogger<AccountController> _logger;

    public AccountController(SignInManager<ApplicationUser> signInManager,
        ILogger<AccountController> logger)
    {
        _signInManager = signInManager;
        _logger = logger;
    }

    [HttpPost("login")]
    [IgnoreAntiforgeryToken]
    public async Task<ActionResult> Login([FromForm] LoginRequest request)
    {
        if (string.IsNullOrEmpty(request.Email) ||
            string.IsNullOrEmpty(request.Password))
            return BadRequest(new { success = false, error = "Vui lòng nhập đầy đủ email và mật khẩu." });

        var result = await _signInManager.PasswordSignInAsync(
            request.Email, request.Password, request.RememberMe, lockoutOnFailure: true);
```

```

        if (result.Succeeded)
        {
            _logger.LogInformation("Người dùng {Email} đăng nhập thành công.", request.Email);
            return Ok(new { success = true, redirect = request.ReturnUrl ?? "/" });
        }
        if (result.IsLockedOut)
            return BadRequest(new { success = false, error = "Tài khoản bị khóa 15 phút do nhập sai 5 lần liên tiếp." });
        if (result.IsNotAllowed)
            return BadRequest(new { success = false, error = "Tài khoản chưa được kích hoạt." });

        return BadRequest(new { success = false, error = "Email hoặc mật khẩu không chính xác." });
    }

    [HttpPost("logout")]
    [Authorize]
    [IgnoreAntiforgeryToken]
    public async Task<ActionResult> Logout()
    {
        await _signInManager.SignOutAsync();
        return Ok(new { success = true });
    }
}

public class LoginRequest
{
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
    public bool RememberMe { get; set; }
    public string? ReturnUrl { get; set; }
}

```

## 7.2 File src\Crm.Web\wwwroot\js\auth-interop.js

```

window.crmAuth = {
    login: async function (email, password, rememberMe, returnUrl) {
        const formData = new FormData();
        formData.append('Email', email);
        formData.append('Password', password);
        formData.append('RememberMe', rememberMe);
        formData.append('ReturnUrl', returnUrl || '/');
    }
}

```

```

    try {
        const response = await fetch('/api/account/login', {
            method: 'POST',
            body: formData,
            credentials: 'include'
        });
        const data = await response.json();
        if (response.ok && data.success) {
            return { success: true, redirect: data.redirect, error: null };
        }
        return { success: false, redirect: null, error: data.error || 'Đăng nhập không thành công.' };
    } catch (err) {
        return { success: false, redirect: null, error: 'Không thể kết nối đến máy chủ xác thực.' };
    }
},

logout: async function () {
    try {
        await fetch('/api/account/logout', {
            method: 'POST',
            credentials: 'include'
        });
    } catch (err) {
        console.error('Logout error:', err);
    } finally {
        window.location.href = '/login';
    }
}
};

```

### 7.3 Trang Đăng nhập src\Crm.Web.Components\Pages\Account\Login.razor

Sử dụng API chuẩn của MudBlazor 8.x (loại bỏ thuộc tính lỗi AdornmentEndIcon):

```

@page "/login"
@layout EmptyLayout
@rendermode InteractiveServer

@inject IJSRuntime JS
@inject NavigationManager Navigation
@inject ILogger<Login> Logger

```

```
<PageTitle>Đăng nhập — Hệ Thống CRM</PageTitle>
```

```

<div class="d-flex align-center justify-center" style="min-height: 100vh; background:
linear-gradient(135deg, #1e3c72 0%, #2a5298 100%);">
  <MudPaper Elevation="8" Class="pa-8" Style="width: 100%; max-width: 440px; border-radius:
12px;">
    <div class="d-flex flex-column align-center mb-6">
      <MudIcon Icon="@Icons.Material.Filled.Shield" Color="Color.Primary" Style="font-size:
56px;" />
      <MudText Typo="Typo.h5" Class="font-weight-bold mt-2">HỆ THỐNG CRM</MudText>
      <MudText Typo="Typo.body2" Color="Color.Secondary">Đăng nhập tài khoản làm
việc</MudText>
    </div>

    <EditForm Model="@_model" OnValidSubmit="HandleLogin">
      <DataAnnotationsValidator />

      <MudTextField @bind-Value="_model.Email"
        Label="Email công ty"
        Variant="Variant.Outlined"
        InputType="InputType.Email"
        Adornment="Adornment.Start"
        AdornmentIcon="@Icons.Material.Filled.Email"
        For="@(() => _model.Email)"
        Disabled="@_isLoading"
        Class="mb-3" />

      <MudTextField @bind-Value="_model.Password"
        Label="Mật khẩu"
        Variant="Variant.Outlined"
        InputType="InputType.Password"
        Adornment="Adornment.Start"
        AdornmentIcon="@Icons.Material.Filled.Lock"
        For="@(() => _model.Password)"
        Disabled="@_isLoading"
        Class="mb-2" />

      <MudCheckBox @bind-Value="_model.RememberMe"
        Label="Ghi nhớ đăng nhập (30 ngày)"
        Color="Color.Primary"
        Class="mb-4" />

      @if (!string.IsNullOrEmpty(_errorMessage))
      {

```

```

        <MudAlert Severity="Severity.Error" Dense="true" Class="mb-4">
            @_errorMessage
        </MudAlert>
    }

    <MudButton ButtonType="ButtonType.Submit"
        Variant="Variant.Filled"
        Color="Color.Primary"
        FullWidth="true"
        Size="Size.Large"
        Disabled="@_isLoading">
        @if (_isLoading)
        {
            <MudProgressCircular Size="Size.Small" Indeterminate="true" Class="mr-2" />
            <span>Đang xác thực...</span>
        }
        else
        {
            <span>Đăng nhập</span>
        }
    </MudButton>
</EditForm>
</MudPaper>
</div>

```

```

@code {
    [SupplyParameterFromQuery]
    public string? returnUrl { get; set; }

    private readonly LoginModel _model = new();
    private bool _isLoading;
    private string? _errorMessage;

    private async Task HandleLogin()
    {
        _isLoading = true;
        _errorMessage = null;

        try
        {
            var res = await JS.InvokeAsync<LoginResult>("crmAuth.login",
                _model.Email, _model.Password, _model.RememberMe, returnUrl ?? "");

```

```

        if (res.Success)
        {
            Navigation.NavigateTo(res.Redirect ?? "/", forceLoad: true);
        }
        else
        {
            _errorMessage = res.Error;
        }
    }
    catch (Exception ex)
    {
        Logger.LogError(ex, "Lỗi đăng nhập.");
        _errorMessage = "Không thể kết nối đến máy chủ. Vui lòng thử lại.";
    }
    finally
    {
        _isLoading = false;
    }
}

private class LoginModel
{
    [System.ComponentModel.DataAnnotations.Required(ErrorMessage = "Email không được để trống")]
    [System.ComponentModel.DataAnnotations.EmailAddress(ErrorMessage = "Định dạng Email không hợp lệ")]
    public string Email { get; set; } = string.Empty;

    [System.ComponentModel.DataAnnotations.Required(ErrorMessage = "Mật khẩu không được để trống")]
    public string Password { get; set; } = string.Empty;

    public bool RememberMe { get; set; }
}

private class LoginResult
{
    public bool Success { get; set; }
    public string? Redirect { get; set; }
    public string? Error { get; set; }
}

```

## 7.4 Cấu hình Router src\Crm.Web\Components\Routes.razor

```
@using Microsoft.AspNetCore.Components.Authorization
@using Crm.Web.Components.Shared
@using Crm.Web.Components.Layout

<Router AppAssembly="@typeof(Program).Assembly">
  <Found Context="routeData">
    <AuthorizeRouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)">
      <NotAuthorized>
        @if (context.User.Identity?.IsAuthenticated != true)
        {
          <RedirectToLogin />
        }
        else
        {
          <MudAlert Severity="Severity.Error">Bạn không có quyền truy cập trang
này.</MudAlert>
        }
      </NotAuthorized>
    </AuthorizeRouteView>
    <FocusOnNavigate RouteData="@routeData" Selector="h1" />
  </Found>
</Router>
```

## 3. BẢNG TỔNG KẾT 10 LỖI THỰC TẾ & BIỆN PHÁP XỬ LÝ (POST-MORTEM)

| # | Mã lỗi / Triệu chứng                                 | Nguyên nhân kỹ thuật                 | Biện pháp xử lý chuẩn hóa                                                                       |
|---|------------------------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------------|
| 1 | CS0234: Microsoft.AspNetCore.Identity does not exist | Crm.Domain chưa reference gói Store. | Chạy dotnet add package Microsoft.Extensions.Identity.Stores trực tiếp trên project Crm.Domain. |

|   |                                                                   |                                                                      |                                                                                                         |
|---|-------------------------------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 2 | CS0718: 'DbSeeder': static types cannot be used as type arguments | Khai báo ILogger<DbSeeder> trên class static.                        | Sử dụng ILoggerFactory.CreateLogger("DbSeeder").                                                        |
| 3 | Warning: UserProfile.UserId1 created in shadow state              | Khai báo HasOne<ApplicationUser>() không chỉ rõ navigation property. | Khai báo public virtual ApplicationUser { get; set; } = null!; trong entity và map HasOne(u => u.User). |
| 4 | column "UserId" of relation "user_profiles" does not exist        | EF Core truy vấn cột theo tên PascalCase, DB lưu snake_case.         | Bổ sung HasColumnName("user_id") cho từng thuộc tính trong OnModelCreating.                             |
| 5 | column "id" referenced in FK constraint does not exist            | DDL script tham chiếu REFERENCES "AspNetUsers"(id) (chữ thường).     | Đổi thành REFERENCES "AspNetUsers"("Id") (chữ I hoa có nháy kép).                                       |
| 6 | relation "aspnetroles" does not exist                             | PowerShell nuốt dấu nháy kép " khi chạy inline psql -c.              | Luôn lưu câu lệnh vào file .sql và chạy bằng tham số -f.                                                |
| 7 | MUD0002: Illegal Attribute 'AdornmentEndIcon'                     | MudBlazor 8.x đã thay đổi API Input.                                 | Chuyển sang dùng InputType.Password mặc định, bỏ logic toggle phức tạp.                                 |
| 8 | AppendOuterContent not found                                      | Thư viện Razor sử dụng cú pháp MudBlazor 6/7 cũ.                     | Cập nhật toàn bộ component lên cú pháp MudBlazor 8.x.                                                   |
| 9 | AmbiguousMatchException: / matched multiple endpoints             | Có nhiều component cùng đăng ký route @page "/".                     | Xóa bỏ các file template mặc định (Counter.razor, Weather.razor,                                        |

|    |                                                    |                                                                 |                                                                                 |
|----|----------------------------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------|
|    |                                                    |                                                                 | Index.razor).                                                                   |
| 10 | Xung đột tên component giữa MudBlazor & ApexCharts | Nạp đồng thời hai thư viện vào Phase 1 khi chưa cần dùng chart. | Tháo hoàn toàn package và config ApexCharts ra khỏi Ngày 3, chuyển sang Ngày 5. |

## 4. KẾT QUẢ NGHIỆM THU HỆ THỐNG

### 4.1 Cơ sở dữ liệu (PostgreSQL)

- **Tổng số bảng:** Đạt đúng **25 bảng** (16 bảng nghiệp vụ Ngày 2 + 7 bảng ASP.NET\* + teams + user\_profiles).
- **Foreign Keys:** 5 ràng buộc từ các bảng nghiệp vụ tới "AspNetUsers"("Id") ở trạng thái hợp lệ.

### 4.2 Seed Data

- **Roles:** Đủ 4 quyền: ADMIN, MANAGER, SALES, ACCOUNTANT.
- **Accounts đã kiểm thử:**

| Vai trò (Role) | Email                | Password     | Mục đích kiểm thử       |
|----------------|----------------------|--------------|-------------------------|
| ADMIN          | admin@crm.local      | Admin@2026   | Quản trị toàn hệ thống  |
| SALES          | sales1@crm.local     | Sales@2026   | Nhân viên kinh doanh 01 |
| SALES          | sales2@crm.local     | Sales@2026   | Nhân viên kinh doanh 02 |
| MANAGER        | manager@crm.local    | Manager@2026 | Quản lý đội ngũ Sales   |
| ACCOUNTANT     | accountant@crm.local | Acc@2026     | Kế toán duyệt báo giá   |

### 4.3 Luồng ứng dụng

- Chưa đăng nhập truy cập / → Tự động chuyển hướng đến /login?returnUrl=%2F.

- Đăng nhập sai 5 lần → Kích hoạt Account Lockout trong 15 phút.
- Đăng xuất qua /logout → Hủy Cookie thành công và đưa về /login.

## 5. KẾ HOẠCH BÀN GIAO SANG NGÀY 4

1. **Scaffold Database-First cho 16 bảng nghiệp vụ:**
  - Sử dụng flag lọc --table để **loại trừ** 7 bảng AspNet\*, user\_profiles và teams nhằm chống ghi đè code Ngày 3.
2. **Kích hoạt DbSets:** Mở comment toàn bộ các thực thể nghiệp vụ trong ApplicationDbContext.
3. **Cấu hình UTC Converter:** Đảm bảo toàn bộ trường DateTime mapping chuẩn TIMESTAMPTZ.
4. **Seed dữ liệu nghiệp vụ:** Tạo dữ liệu demo cho Khách hàng, Cơ hội và Báo giá.